



Практическо изпитване 5

Модул 1: Обектно ориентирано програмиране

Момчетата решават вариант 3, а момичетата – вариант 4. В `Main()` да присъстват само имената на извиканите методи! Без шльоковица! Предайте `Program.cs` файла и един скрийншот на резултата от терминала в Classroom.

Вариант 3 за момчета: Дядо Коледа и подготовката на шейната

Задача: Дядо Коледа проверява и подготвя шейната за коледарската нощ. Всички части на шейната имат общи характеристики, но някои са за движение, а други - за комфорт.

1. Създайте абстрактен клас `SleighPart` със следните полета:
 - `PartName` (име на частта)
 - `Material` (материал: "дърво", "метал", "плат")
 - `Weight` (тегло в кг)
2. Създайте два наследника на `SleighPart`:
 - Клас `MovementPart` с допълнително поле:
 - `SpeedFactor` (коефициент за скорост от 1 до 10)
 - Клас `ComfortPart` с допълнително поле:
 - `SoftnessLevel` (ниво на мекота от 1 до 10)
3. В абстрактния клас `SleighPart` добавете:
 - Статично поле `TotalWeight`, което сумира теглото на всички части.
 - Абстрактен метод `GetFunctionDescription()`, който връща `string` - описание на функцията.
 - Виртуален метод `DisplayInfo()`, който отпечатва: "Част: {PartName} от {Material}."
4. Във всеки наследник (`MovementPart` и `ComfortPart`):
 - Имплементирайте абстрактния метод `GetFunctionDescription()`:
 - За `MovementPart`: "Увеличава скоростта с фактор {SpeedFactor}."
 - За `ComfortPart`: "Осигурява комфорт с ниво {SoftnessLevel}."
 - Презапишете виртуалния метод `DisplayInfo()`, като добавете:
 - За `MovementPart`: "Скоростен фактор: {SpeedFactor}"
 - За `ComfortPart`: "Ниво на мекота: {SoftnessLevel}"
5. В главния метод `main`:
 - Създайте списък (`List<SleighPart>`) с няколко обекта от двата типа.
 - Намерете най-тежката част от списъка.
 - Изчислете средното ниво на мекота на всички `ComfortPart` части.
 - Използвайте търсачка, за да намерите всички части от "дърво".

- Отпечатайте **общото тегло** на всички части (`TotalWeight`).
-

Вариант 4 за момичета: Феите и магията на снежинките

Задача: Зимните феи създават магически снежинки за покриване на Приказния лес. Всяка снежинка има уникални свойства.

1. **Създайте абстрактен клас `MagicSnowflake` със следните полета:**
 - `DesignName` (име на дизайна)
 - `MagicPower` (магическа сила от 1 до 100)
 - `CreationTime` (време за създаване в минути)
 2. **Създайте два наследника на `MagicSnowflake`:**
 - **Клас `CoveringSnowflake` с допълнително поле:**
 - `CoverageArea` (площ за покриване в m^2)
 - **Клас `GlowingSnowflake` с допълнително поле:**
 - `GlowColor` (цвят на светенето)
 3. **В абстрактния клас `MagicSnowflake` добавете:**
 - **Статично поле** `TotalMagicPower`, което сумира магическата сила на всички снежинки.
 - **Абстрактен метод** `GetSpell()`, който връща `string` - магическа формула.
 - **Виртуален метод** `DisplayInfo()`, който отпечатва: "Снежинка '{DesignName}' със сила {MagicPower}."
 4. **Във всеки наследник (`CoveringSnowflake` и `GlowingSnowflake`):**
 - **Имплементирайте абстрактния метод `GetSpell()`:**
 - За `CoveringSnowflake`: "Покривам {CoverageArea} m^2 с бял сняг!"
 - За `GlowingSnowflake`: "Светя с {GlowColor} цвят в нощта!"
 - **Презапишете виртуалния метод `DisplayInfo()`, като добавите:**
 - За `CoveringSnowflake`: "Покрива: {CoverageArea} m^2 "
 - За `GlowingSnowflake`: "Цвят: {GlowColor}"
 5. **В главния метод `main`:**
 - Създайте **списък (`List<MagicSnowflake>`)** с няколко обекта от двата типа.
 - Намерете **снежинката с най-голяма магическа сила**.
 - Изчислете **средната площ за покриване** на `CoveringSnowflake` снежинките.
 - Използвайте **търсачка**, за да намерите всички снежинки, които **светят в синьо**.
 - Отпечатайте **общата магическа сила** на всички снежинки (`TotalMagicPower`).
-